

DEVSECOPS PIPELINE FOR SECURE SOFTWARE DELIVERY PLATFORM

Software Engineering — CO2 Assessment-1

M.Tharani(192472375)

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements of the "DevSecOps Pipeline for Secure Software Delivery Platform." The purpose of this document is to provide a clear, complete, and unambiguous description of the proposed system so that developers, testers, project supervisors, and other stakeholders share a common understanding of what the system must do. This document is prepared in accordance with the IEEE 830 standard for software requirements specifications and is intended to support the Software Engineering CO2 Assessment.

1.2 Project Overview

Modern software teams release updates frequently, but speed of delivery is often achieved at the cost of security. Many organisations still treat security testing as a final, manual step performed just before release, which is slow, error-prone, and easy to skip under deadline pressure. The DevSecOps Pipeline for Secure Software Delivery Platform is designed to solve this problem by embedding automated security checks directly into the Continuous Integration and Continuous Delivery (CI/CD) workflow. Every code commit automatically passes through build, test, static and dynamic security scanning, dependency analysis, container scanning, and controlled deployment stages, with a security gate that stops any release that carries a critical vulnerability.

1.3 Objectives

- To automate the integration of security testing into every stage of the software delivery lifecycle.
- To detect vulnerabilities in source code, dependencies, and container images before deployment.
- To reduce the time and manual effort required to release secure software.
- To enforce consistent security policies through automated quality and security gates.
- To provide traceability and audit evidence for compliance and governance purposes.
- To give stakeholders real-time visibility into the security posture of the software delivery pipeline.

1.4 Scope of the System

The system covers the complete secure software delivery pipeline, starting from source code commit and ending with production deployment and monitoring. It includes source code management

integration, automated build and test execution, static and dynamic security analysis, software composition analysis, container image scanning, secrets management, automated deployment, rollback capability, audit logging, and compliance reporting. The system does not include the development of the application code itself; it operates as a supporting platform that any software project can plug into. Physical network security, employee background checks, and legal compliance auditing outside the software delivery process are outside the scope of this system.

2. SCENARIO ANALYSIS

2.1 Business Domain

The system operates within the software development and IT services domain, specifically targeting organisations that build and deliver web-based and enterprise software products. It is applicable to software houses, banking and fintech companies, healthcare software vendors, and any organisation that must deliver frequent software updates while complying with security and regulatory standards such as OWASP Top 10, ISO/IEC 27001, and GDPR.

2.2 Existing Problems

- Security testing is usually performed manually and only at the end of the development cycle, causing late discovery of vulnerabilities.
- Developers lack immediate feedback on insecure coding practices during development.
- Third-party library vulnerabilities often go unnoticed until after deployment.
- Deployment processes are frequently manual, increasing the risk of human error and inconsistent configurations.
- Lack of centralized audit trails makes compliance reporting slow and unreliable.
- Security and development teams work in silos, which delays remediation of identified issues.

2.3 Proposed Solution

The proposed DevSecOps Pipeline integrates automated security scanning tools directly into the CI/CD workflow so that every commit is automatically built, tested, and scanned before it can progress toward deployment. A configurable security gate blocks any build that contains high-severity vulnerabilities, ensuring that insecure code cannot reach production. The platform also provides a centralized

dashboard, automated notifications, and audit logging so that all stakeholders have real-time visibility into the health and security status of the pipeline.

2.4 Stakeholders

#	Stakeholder	Role / Interest
1	Software Developers	Commit code, receive early feedback on code quality and vulnerabilities.
2	Security Officers	Define security policies, review scan results, approve exceptions.
3	DevOps Engineers	Configure and maintain pipeline infrastructure and deployment automation.
4	Quality Assurance Engineers	Configure and monitor automated test suites.
5	Project Managers	Track release progress, review compliance and status reports.
6	End Users / Customers	Benefit indirectly from more secure and reliable software releases.
7	Compliance Auditors	Use audit logs and reports to verify regulatory compliance.

2.5 System Objectives

- Shorten the feedback loop between code commit and vulnerability detection.
- Prevent insecure builds from reaching production environments.
- Standardize security practices across all software projects in the organisation.
- Provide measurable, auditable evidence of secure development practices.

3. FUNCTIONAL REQUIREMENTS

The table below lists the functional requirements of the DevSecOps Pipeline. Each requirement is uniquely identified and assigned a priority level to guide implementation planning.

Req. ID	Requirement	Description	Priority
FR-01	User Authentication	The system shall allow developers, security officers, and administrators to log in using secure credentials and multi-factor authentication.	High
FR-02	Role-Based Access Control	The system shall restrict access to pipeline stages, source code, and deployment controls based on assigned user roles.	High
FR-03	Source Code Repository Integration	The system shall integrate with version control systems (e.g., Git) to detect code commits and trigger pipeline execution automatically.	High
FR-04	Automated Build Management	The system shall compile and package source code automatically whenever a new commit or pull request is received.	High
FR-05	Static Application Security Testing (SAST)	The system shall scan source code for known vulnerabilities and insecure coding patterns before the build stage completes.	High
FR-06	Software Composition Analysis (SCA)	The system shall scan third-party libraries and dependencies for known vulnerabilities (CVEs) and license risks.	High
FR-07	Dynamic Application Security Testing (DAST)	The system shall perform automated security testing on a running instance of the application in a staging environment.	Medium
FR-08	Automated Unit and Integration Testing	The system shall execute predefined unit and integration test suites automatically after every build.	High

Req. ID	Requirement	Description	Priority
FR-09	Container Image Scanning	The system shall scan container images for vulnerabilities and misconfigurations before pushing them to the registry.	High
FR-10	Security Gate Enforcement	The system shall block the pipeline from proceeding to the next stage if a security scan identifies critical or high-severity vulnerabilities.	High
FR-11	Secrets Management	The system shall securely store and inject credentials, API keys, and certificates into the pipeline without exposing them in logs or code.	High
FR-12	Automated Deployment	The system shall automatically deploy verified builds to staging and production environments upon successful completion of all quality and security gates.	High
FR-13	Rollback Mechanism	The system shall provide an automated rollback to the last stable release if a deployment fails or a critical issue is detected post-deployment.	Medium
FR-14	Audit Logging	The system shall record all pipeline activities, user actions, and scan results in a tamper-evident audit log.	High
FR-15	Real-Time Notifications	The system shall notify relevant stakeholders through email or chat integration (e.g., Slack) when a pipeline stage fails or a vulnerability is found.	Medium
FR-16	Compliance Reporting	The system shall generate reports mapping scan results and pipeline activities to compliance standards such as OWASP and ISO 27001.	Medium
FR-17	Dashboard and Visualization	The system shall provide a centralized dashboard displaying pipeline status, security posture, and historical trends.	Medium

Req. ID	Requirement	Description	Priority
FR-18	Configuration as Code	The system shall allow pipeline configurations to be defined, version-controlled, and reused as code (e.g., YAML files).	Low
FR-19	Environment Provisioning	The system shall support automated provisioning and de-provisioning of test and staging environments using infrastructure-as-code tools.	Medium
FR-20	Multi-Project Support	The system shall support the configuration and management of pipelines for multiple independent software projects.	Low

4. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements define the quality attributes that the DevSecOps Pipeline must satisfy in addition to its functional behaviour. These requirements are critical because they directly affect the trustworthiness, adoption, and long-term sustainability of the platform.

4.1 Performance

The pipeline shall complete a standard build, test, and scan cycle for a medium-sized application (approximately 50,000 lines of code) within 15 minutes under normal load. Static and dependency scans shall not add more than 30% overhead to the total pipeline execution time.

4.2 Security

All communications between pipeline components shall be encrypted using TLS 1.2 or higher. Secrets and credentials shall never be stored in plain text or exposed in build logs. Access to pipeline configuration and deployment controls shall require authentication and be restricted by role-based permissions.

4.3 Reliability

The pipeline shall correctly execute all configured stages in 99% of pipeline runs, excluding failures caused by external systems outside the platform's control. Failed stages shall produce clear, actionable error messages and shall not leave the system in an inconsistent state.

4.4 Availability

The core pipeline orchestration service shall be available at least 99.5% of the time, excluding scheduled maintenance windows, to support continuous development activity across multiple time zones.

4.5 Scalability

The system shall support a growing number of concurrent pipeline executions and additional projects without requiring architectural redesign. It shall support horizontal scaling of build and scanning agents to handle increased workload during peak development periods.

4.6 Usability

The dashboard and configuration interfaces shall be intuitive enough for a developer with basic CI/CD experience to configure a new pipeline within one working day, supported by clear documentation and inline guidance.

4.7 Maintainability

The system shall be built using modular, loosely coupled components so that individual tools (for example, a specific scanning engine) can be replaced or upgraded without requiring changes to the rest of the pipeline. Configuration shall be managed as version-controlled code to simplify maintenance and rollback of pipeline definitions.

5. ACTORS AND USE CASES

5.1 Primary and Secondary Actors

Primary actors are the human roles that directly interact with the platform. Secondary actors are external systems that the platform integrates with automatically.

Actor	Type	Description
Developer	Primary	Commits source code, configures pipelines, and views build/scan results.
Security Officer	Primary	Defines security policies, reviews scan findings, manages security gate rules.
DevOps Engineer	Primary	Maintains pipeline infrastructure, manages deployments and rollbacks.
Quality Assurance Engineer	Primary	Configures automated test suites and reviews test outcomes.
Project Manager	Primary	Approves releases and monitors compliance reports and dashboards.
Version Control System	Secondary (external)	Supplies source code commits that trigger pipeline execution.
Vulnerability Database	Secondary (external)	Supplies up-to-date CVE data used during dependency scanning.
Container Registry	Secondary (external)	Stores and receives scanned container images.
Notification Service	Secondary (external)	Delivers automated alerts (e.g., email, chat) to stakeholders.

5.2 Major Use Cases by Actor

Developer: Commit Source Code, Trigger Automated Build, Configure Pipeline as Code, View Dashboard.

Security Officer: Run Static Analysis, Run Dependency Scan, Run Dynamic Analysis, Enforce Security Gate, Manage Secrets, Generate Compliance Report.

DevOps Engineer: Trigger Automated Build, Scan Container Image, Deploy to Staging/Production, Rollback Release, Configure Pipeline as Code.

Quality Assurance Engineer: Run Unit/Integration Tests, Run Dynamic Analysis, View Dashboard.

Project Manager: View Dashboard, Generate Compliance Report, Approve Release.

5.3 UML Use Case Diagram (PlantUML)

The PlantUML script below describes the complete use case diagram for the system, including all primary and secondary actors and their associated use cases. The script can be rendered using any PlantUML-compatible tool (for example, the PlantUML online server or the PlantUML VS Code extension).

```
@startuml
left to right direction
skinparam packageStyle rectangle
actor "Developer" as Dev
actor "Security Officer" as Sec
actor "DevOps Engineer" as Ops
actor "Project Manager" as PM
actor "Quality Assurance Engineer" as QA
actor "Version Control System" as VCS <<external>>
actor "Vulnerability Database" as CVE <<external>>
actor "Notification Service" as Notify <<external>>
actor "Container Registry" as Reg <<external>>

rectangle "DevSecOps Pipeline for Secure Software Delivery" {
    usecase "Commit Source Code" as UC1
    usecase "Trigger Automated Build" as UC2
    usecase "Run Static Analysis (SAST)" as UC3
    usecase "Run Dependency Scan (SCA)" as UC4
    usecase "Run Unit/Integration Tests" as UC5
    usecase "Run Dynamic Analysis (DAST)" as UC6
    usecase "Scan Container Image" as UC7
    usecase "Enforce Security Gate" as UC8
    usecase "Manage Secrets" as UC9
    usecase "Deploy to Staging/Production" as UC10
    usecase "Rollback Release" as UC11
    usecase "View Dashboard" as UC12
}
```

```
usecase "Generate Compliance Report" as UC13
usecase "Configure Pipeline as Code" as UC14
usecase "Approve Release" as UC15
usecase "Receive Notifications" as UC16
}
```

Dev --> UC1

Dev --> UC2

Dev --> UC14

Dev --> UC12

Sec --> UC3

Sec --> UC4

Sec --> UC6

Sec --> UC8

Sec --> UC9

Sec --> UC13

Ops --> UC2

Ops --> UC7

Ops --> UC10

Ops --> UC11

Ops --> UC14

QA --> UC5

QA --> UC6

QA --> UC12

PM --> UC12

PM --> UC13

PM --> UC15

UC1 --> UC2 : <<include>>

```
UC2 --> UC3 : <<include>>
UC3 --> UC4 : <<include>>
UC4 --> UC5 : <<include>>
UC5 --> UC6 : <<include>>
UC6 --> UC7 : <<include>>
UC7 --> UC8 : <<include>>
UC8 --> UC10 : <<extend>>
UC10 --> UC11 : <<extend>>
```

```
VCS --> UC1
CVE --> UC4
Reg --> UC7
Notify --> UC16
@enduml
```

6. SOFTWARE REQUIREMENT SPECIFICATION (SRS)

6.1 Problem Description

Organisations that release software frequently struggle to maintain security without slowing down delivery. Manual, end-of-cycle security reviews cannot keep pace with continuous deployment practices, resulting in vulnerabilities reaching production, delayed releases, and inconsistent enforcement of security policy across teams. There is a need for a platform that automatically embeds security verification into every step of the delivery pipeline without requiring developers to become security experts.

6.2 System Features

- Automated CI/CD orchestration triggered directly by source code commits.
- Integrated static, dynamic, and dependency vulnerability scanning.
- Configurable security gates that automatically block insecure builds.
- Secure secrets management with encrypted storage and controlled injection.
- Automated deployment with rollback support for staging and production environments.

- Centralized dashboard for real-time pipeline and security status visibility.
- Audit logging and compliance report generation for governance purposes.

6.3 Data Requirements

The system shall store and manage the following categories of data:

Data Entity	Description	Examples of Attributes
Pipeline Configuration	Defines the stages, tools, and rules for a project's pipeline.	Project ID, stage sequence, tool settings, gate thresholds
Build and Scan Results	Stores outcomes of build, test, and security scans.	Build ID, timestamp, status, vulnerability list, severity
User and Role Data	Stores user accounts and their access permissions.	User ID, role, permissions, authentication details
Audit Logs	Records all significant actions performed within the system.	Event ID, actor, action, timestamp, outcome
Secrets Metadata	Stores references to securely vaulted credentials (not the secrets themselves in plain text).	Secret ID, owner, associated pipeline, expiry date

6.4 External Interface Requirements

6.4.1 User Interfaces

The system shall provide a web-based dashboard accessible through standard modern browsers (Chrome, Firefox, Edge). The interface shall present pipeline status, scan results, and compliance reports using clear visual indicators such as colour-coded severity levels and progress bars.

6.4.2 Software Interfaces

The system shall integrate with version control platforms (e.g., Git-based systems), container registries, scanning engines (SAST, DAST, SCA tools), and notification services (e.g., email servers, chat platforms) through standard REST APIs and webhooks.

6.4.3 Hardware Interfaces

The system shall operate on standard server and cloud virtual machine infrastructure and does not require any specialised or proprietary hardware. Build and scan agents may be deployed on containerised infrastructure to enable elastic scaling.

6.4.4 Communication Interfaces

All communication between the pipeline components, external tools, and the dashboard shall occur over HTTPS. Internal service-to-service communication shall use encrypted channels, and webhook payloads shall be authenticated to prevent spoofed trigger events.

7. ASSUMPTIONS AND CONSTRAINTS

7.1 Assumptions

- All development teams use a supported version control system for source code management.
- Projects onboarded to the pipeline follow a consistent branching and commit workflow.
- Adequate compute infrastructure is available to run build and scanning agents.
- Stakeholders have basic familiarity with CI/CD concepts and terminology.

7.2 Technical Constraints

- The system must integrate with existing open-source and commercial scanning tools rather than building all scanning engines from scratch.
- The platform must be compatible with containerised deployment environments.
- Pipeline configuration must be expressible in a human-readable, version-controllable format such as YAML.

7.3 Operational Constraints

- The platform must operate continuously to support development activity across different working hours and time zones.
- Downtime for maintenance must be scheduled and communicated in advance to avoid disrupting active releases.

7.4 Time Constraints

- The project must be designed, documented, and evaluated within the academic semester timeline defined for the CO2 Assessment.
- Pipeline execution time targets (see Section 4.1) must be met to remain practical for daily development use.

7.5 Security Constraints

- All secrets and credentials must be stored using encryption at rest and in transit.
- The system must enforce the principle of least privilege for all user roles.
- No build shall be permitted to reach production while unresolved critical vulnerabilities exist.

8. REQUIREMENT VALIDATION

8.1 Completeness

All requirements identified during scenario analysis have been translated into functional or non-functional requirements. Each actor identified in Section 5 has at least one associated use case, and every functional requirement maps to a specific stage of the software delivery pipeline described in the project overview.

8.2 Consistency

Requirements have been cross-checked against one another to confirm that no two requirements conflict. For example, the requirement for automated deployment (FR-12) is consistent with the security gate requirement (FR-10), since deployment is only triggered after the gate has been successfully passed.

8.3 No Ambiguity

Each requirement uses precise, measurable, and testable language. Vague terms such as "fast" or "secure" have been replaced with specific criteria, for example defining performance targets in minutes and specifying encryption standards such as TLS 1.2, so that each requirement can be objectively verified.

8.4 No Redundancy

Each functional requirement addresses a distinct capability of the system. Overlapping concerns, such as scanning at different stages (static, dynamic, dependency, and container scanning), have been kept

separate because they test different aspects of the system, and no single requirement duplicates the intent of another.

8.5 Suggested Improvements

- Introduce automated risk-scoring to prioritise which vulnerabilities require immediate attention.
- Add support for progressive delivery strategies such as canary releases to further reduce deployment risk.
- Extend compliance reporting to support additional standards as organisational needs evolve.

9. CONCLUSION

This Software Requirements Specification has defined the functional and non-functional requirements, actors, use cases, and constraints of the DevSecOps Pipeline for Secure Software Delivery Platform. By embedding automated security testing directly into the CI/CD workflow and enforcing security gates before deployment, the proposed system addresses the common problem of late-stage, manual security reviews. The result is a software delivery process that is faster, more consistent, and demonstrably more secure, giving development teams the confidence to release frequently without compromising the safety of their users or the integrity of their systems. The requirements documented here provide a solid foundation for the design, implementation, and evaluation phases of this project.